



Zeal Institute – Project Management System

Digital solution to manage the end-to-end academic project lifecycle for ZIBACAR (affiliated to SPPU, AICTE approved, DTE recognized).

Presented by: Omkar Dhumal

Nikhil Devkar

Why a Digital System?

Current Challenges

Manual tracking, scattered documents, missed deadlines, and high admin workload.

Opportunity

Centralize monitoring, automate notifications, and enforce stage-wise progress tracking.





System Architecture

Three-tier architecture: Presentation (UI/dashboards), Application (Flask backend, business logic), Data (MongoDB collections for users, batches, stages, submissions, notifications).

Primary Roles & Permissions



Admin (Coordinator)

Create/manage batches, assign mentors, bulk upload students, configure stages/deadlines, monitor progress.



Faculty (Mentor)

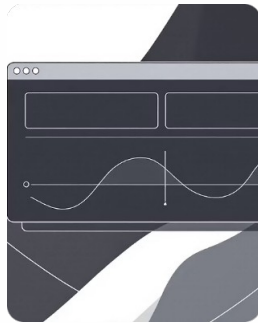
View assigned students, review & approve submissions, add remarks, track stage-wise progress.



Student (Mentee)

Upload documents, view mentor, track deadlines and statuses, receive notifications.

Scope – Core Functionalities



Batch & Stage Management

Create/reorder stages, set deadlines, and monitor batch progress.



Submission Workflow

Upload, review, approve/reject with remarks; support PDF/DOC/PPT and replacements before approval.



Notifications

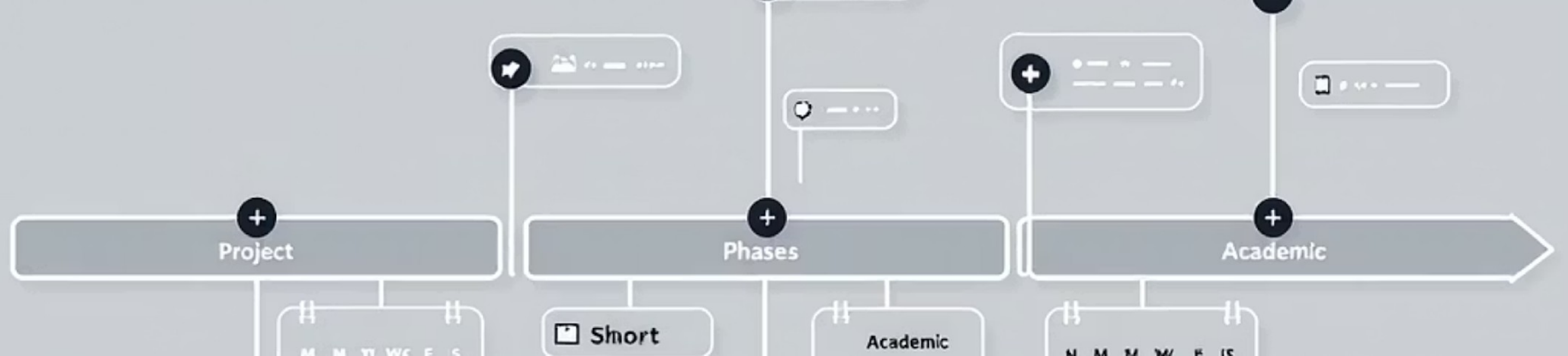
Automated email and in-app alerts for deadlines, approvals, and late submissions.

Technical Stack



- Frontend: HTML5, CSS3, JavaScript (ES6)
- Backend: Python (Flask)
- Database: MongoDB (NoSQL)
- Tools: VS Code, MongoDB Compass, modern browsers

Open-source stack ensures economic feasibility and easy deployment.



Feasibility Overview

Operational

User-friendly, minimal training; browser-based access for students and faculty.

Technical

Proven open-source technologies; modular design for scalability.

Economic

No licensing costs; deployable on free or low-cost hosting.

Time

Modular scope allows completion within an academic project timeline.

Functional & Non-Functional Requirements

Key Functional Needs

- Secure login & role-based access
- Bulk student upload (Excel)
- Stage-wise submission & approval
- Deadline monitoring & late detection

Non-Functional Standards

- Performance: fast load, efficient uploads
- Usability: responsive, intuitive UI
- Security: authenticated access, secure file storage
- Compatibility: desktop/tablet/mobile, modern browsers

Operating Environment & Hardware



Minimum hardware: PC/laptop, 4 GB RAM, 20 GB free storage, stable internet.

Software environment: Windows OS, Chrome/Firefox/Edge, Flask backend, MongoDB database, VS Code for development.

Expected Benefits & Next Steps

Benefits

Centralized monitoring, reduced admin effort, transparent stage tracking, timely notifications.

Next Steps

Finalize requirements, develop modular sprints, deploy to cloud, pilot with one batch, iterate on feedback.

